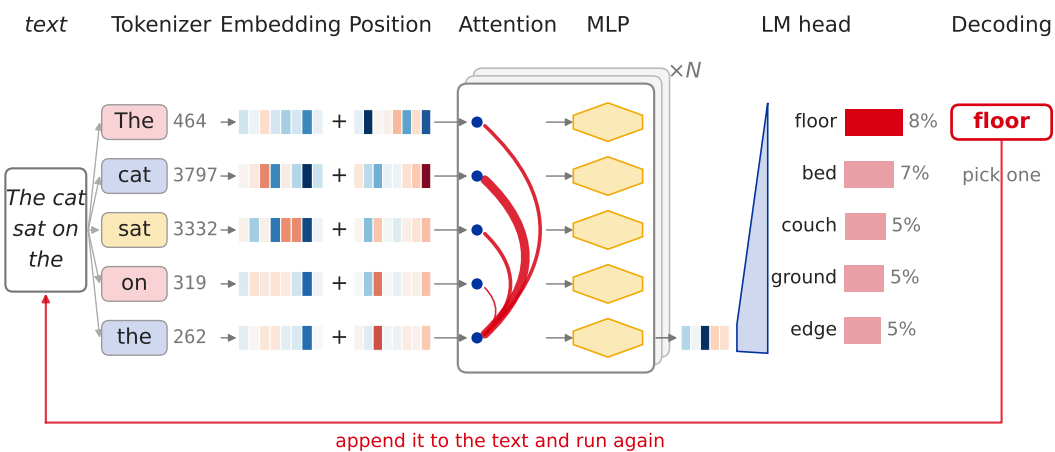
Quantization

An LLM on your laptop

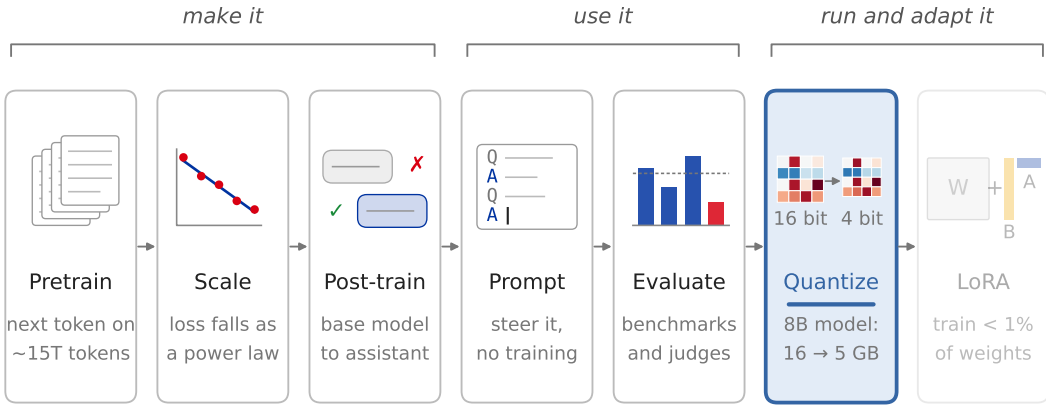
LLM chapter, session 12. Toy numbers unless a slide says otherwise.

Where we are: one forward pass



Every box of the pass multiplies by stored weights. Today: store them in fewer bits.

Where we are: the life of a model



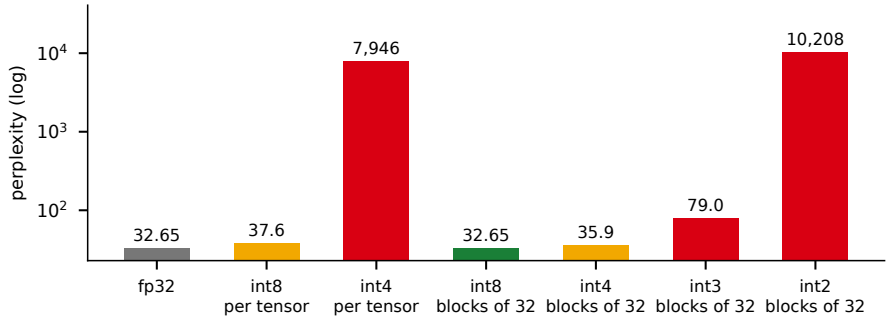
Today: the first box of "run and adapt it".

Does it fit?

- An 8B model in bf16: $8 \times 10^9 \times 2$ bytes $= 16$ GB.
- This laptop: 16 GB of memory - for everything, the operating system included.
- At 4 bits per weight it would be about 4 GB (4.5 GB with the scales; real 4-bit files keep a few tensors at higher precision, so the map rounds to 5).

Predict first: store every weight in 4 bits instead of 16 or 32. How much worse does the model get - a little, or completely?

GPT-2 at 4 bits, two ways



Real: GPT-2's 48 block weight matrices rounded in NumPy, perplexity measured with the cached model on 8,192 tokens of wikitext (LLM-7's perplexity; lower is better). One scale per matrix at 4 bits: 7,946 - destroyed. A scale per block of 32 weights: 35.9, against 32.65 in fp32.

Answer: both. Same 4 bits, opposite results; even 8 bits with one scale (37.6) loses to 4 bits in blocks. A **scale** maps the stored integers back to real numbers, and where it comes from matters more than the bit count - this lecture's main idea.

Outline

The bill

Fewer bits

Smarter than rounding

Running it

The bill

Every weight and every cached token costs bytes.

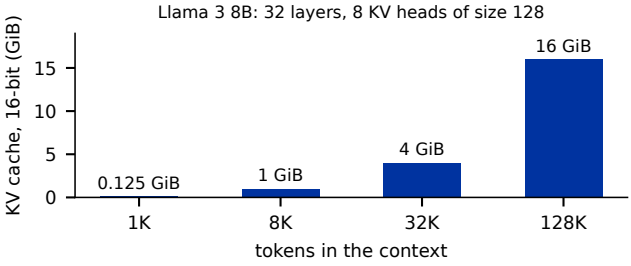
Memory, by hand

Weights: parameters $\times$ bytes per parameter.

	GPT-2 small	**an 8B model**
parameters	124,439,808	8×10^9
fp32, 4 bytes	498 MB	32 GB
bf16, 2 bytes	249 MB	16 GB
4.5 bits (4-bit blocks)	70 MB	4.5 GB

Training needed about 16 bytes per parameter (LLM-8). Running needs only the weights - and they can shrink further.

The KV cache



Generating, the model keeps every earlier token's keys and values (LLM-3) so it does not recompute them. Per token:

$2 \times$ layers $\times$ KV heads $\times$ head size $\times$ bytes

(the 2: one key and one value).

By hand, 16-bit: GPT-2, $2 \times 12 \times 768 \times 2 = 36{,}864$ bytes per token, 37.7 MB for 1,024 tokens. Llama 3 8B, $2 \times 32 \times 8 \times 128 \times 2 = 131{,}072$ bytes (128 KiB): 1 GiB at 8K tokens, 16 GiB at 128K.

Long context costs memory too - at 128K about the size of the whole 16-bit model. Llama 3 keeps 8 key/value heads, not 32: 4× less. How that works: LLM-14.

Fewer bytes is also faster

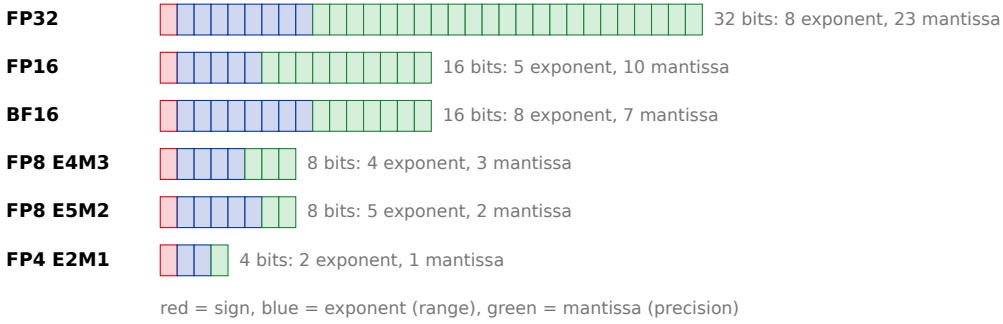
- Generating one token reads every weight once (one user; a dense model, where every weight is used for every token). The arithmetic is quick; moving the bytes is the slow part.
- So: tokens per second $\leq$ memory bandwidth (bytes per second the memory delivers) / bytes read per token.
- **Measured** on this laptop: a NumPy copy moves 15.7 GB/s, counting bytes read and written - an estimate. An 8B model at 16 GB (it would not even fit): at most 1.0 tokens/s. At 4.5 GB: at most 3.5 tokens/s. (The KV cache adds a little to each read.)

Quantizing to 4 bits does not just make the model fit; it makes it up to about 3.5$\times$ faster to generate on the same machine.

Fewer bits

What a number costs, and what rounding costs.

Number formats



- A float stores a sign, an exponent (how big: the range) and a mantissa (how precise). BF16 keeps FP32's 8 exponent bits and drops precision; FP16 keeps more precision and loses range.
- Integers (INT8, INT4) have no exponent: one shared **scale** says how big the integers are. Everything today is about choosing that scale.

Absmax, by hand

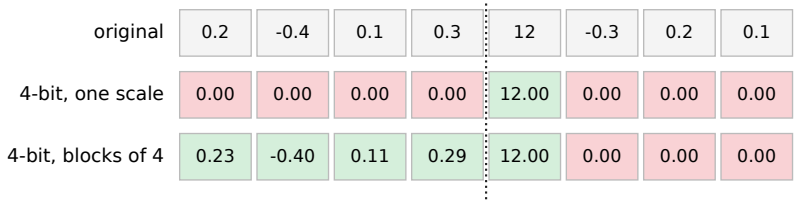
Four weights: $[0.5, -1.1, 0.3, 2.4]$. Pick the scale so the largest becomes the largest integer:

	scale	**integers**	**back to numbers**
INT8 (to ± 127)	$2.4/127 = 0.0189$	$[26, -58, 16, 127]$	$[0.49, -1.10, 0.30, 2.40]$
INT4 (to ± 7)	$2.4/7 = 0.343$	$[1, -3, 1, 7]$	$[0.34, -1.03, 0.34, 2.40]$

integer $=$ round(w/scale); back $=$ integer $\times$ scale. Store the integers and one scale.

At 4 bits there are only 15 levels (-7 to 7), so every weight moves to its nearest level: $0.5 \to 0.34$. The largest weight sets the step size for all of them.

One outlier



Toy numbers. One weight is 12.0, the others below 0.5. With one scale for all eight, the step is $12/7 = 1.7$, so every small weight rounds to 0 (red; green = kept). With a scale per block of 4 (dotted line), only the block that holds the outlier suffers.

That is the cold open: a few large *weights* in each GPT-2 matrix, and one scale per matrix zeroes out most of the rest. Blocks keep the damage local.

Blocks, and their price

- Each block stores its own scale, usually 16 bits. Bits per weight: $4 + 16/32 =$ **4.5** for 4-bit blocks of 32.
- GPT-2, blocks of 32: 8 bits - no loss (32.65 both); 4 bits - 35.9; 3 bits - 79.0; 2 bits - 10,208.
- Hardware formats bake this in. **MXFP4** (an open standard): blocks of 32 4-bit floats with an 8-bit power-of-two scale. **NVFP4** (NVIDIA): blocks of 16 with an FP8 scale, plus one scale for the whole tensor.

Smaller blocks fit the local numbers better and cost more scale bits. Around 4 bits per weight is where most local models run today.

Outliers in real models

- A second kind of outlier, in the *activations* (the values flowing between layers, not the stored weights): in large models a few dimensions are huge. Dettmers et al. (2022): handle those dimensions in 16 bits and "more than 99.9% of values" can still be multiplied in 8 bits (LLM.int8()).
- Activations change with every input; weights are fixed and can be studied once.
- So the common path on laptops: quantize the **weights** only, keep the activations in 16 bits. Each 4-bit weight is turned back into 16 bits (integer $\times$ scale) just before it is used; it is stored in 4.

Weight-only quantization saves memory and bandwidth - the two things that limit a laptop. The arithmetic still happens in 16 bits.

Smarter than rounding

Fix the errors rounding makes.

Correct the error

- Rounding treats each weight alone. But a layer's output is a sum: if one weight rounds down, others can be nudged to make up for it.
- **GPTQ** (Frantar et al., 2022): quantize a layer one column at a time, and after each step adjust the not-yet-quantized weights to cancel the error, using a small sample of real inputs (a calibration set).
- **AWQ** (Lin et al., 2023): find the input channels that carry large activations and protect their weights by scaling them up before rounding.

Both quantize a finished model (PTQ, "post-training quantization" - not LLM-9's post-training): no retraining, minutes to hours. The details are optional; the idea is "round, then repair".

Train for it

- **Quantization-aware training** (QAT): during training, round the weights in the forward pass, so the model learns to live with the rounding.
- gpt-oss (OpenAI, 2025) was trained with its expert weights in MXFP4. It is a mixture of experts: 21B parameters stored, only 3.6B used per token. It runs "within 16GB of memory".
- At the extreme: BitNet b1.58 trains ternary weights - only -1, 0 or $+1$, $\log_2 3 = 1.58$ bits each - from scratch.

The best 4-bit model is one that was trained to be 4-bit. Rounding after the fact is the cheaper, more common path.

Running it

The program between the file and your question.

On this laptop, by hand

An 8B model at 4.5 bits per weight, on 16 GB:

weights, $8 \times 10^9 \times 4.5/8$ bytes	4.5 GB
KV cache, 8K tokens of context (16-bit)	1 GiB
operating system, browser, the app	a few GB
total	fits, with room

Speed: at most $15.7/4.5 \approx 3.5$ tokens per second from the measured bandwidth - readable, if not fast. In bf16 the same model would not load at all.

That is the whole lecture in one table: fewer bits made it fit and made it faster.

What an inference engine does

A quantized file is just numbers. An **inference engine** is the program that turns it into answers. It exists because each of these is hard to do well:

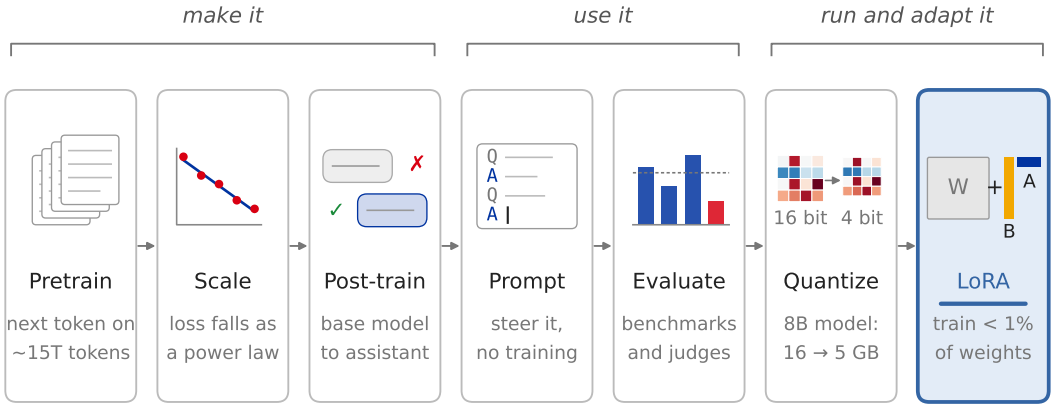
- load the weights in their packed format, and multiply with them directly;
- keep and reuse the KV cache, and drop old entries when memory runs out;
- decode (LLM-6) and stream tokens as they come;
- serve many users at once by running their requests together (batching);
- offer an API that applications already know.

Laptop engines optimise for one user on a CPU or a small GPU; server engines for many users on big GPUs. Names, as of October 2026: llama.cpp, Ollama and LM Studio on laptops; vLLM and SGLang on servers.

Recap

- **The bill**: weights cost parameters $\times$ bytes; the KV cache costs bytes per token and grows with the context. Generation is limited by memory bandwidth, so fewer bytes is also faster.
- **Fewer bits**: store integers and a scale. One scale per matrix lets one outlier ruin it (GPT-2 at 4 bits: 7,946); a scale per block of 32 keeps it (35.9).
- **Smarter than rounding**: correct the error (GPTQ, AWQ) or train for it (QAT, gpt-oss in MXFP4, ternary BitNet).
- **Running it**: an inference engine loads the packed weights, manages the cache and serves the answers.

Next on the map



Next (LLM-13): adapting a model on a budget - LoRA, and QLoRA on a 4-bit base.

Sources & further reading

- Dettmers et al. (2022), "LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale".
- Frantar et al. (2022), "GPTQ"; Lin et al. (2023), "AWQ: Activation-aware Weight Quantization".
- Open Compute Project, "Microscaling Formats (MX) Specification"; NVIDIA, NVFP4.
- OpenAI (2025), gpt-oss model card; Ma et al. (2024), "The Era of 1-bit LLMs" (BitNet b1.58).
- Llama Team (2024), "The Llama 3 Herd of Models" (the 8B configuration).

Figures: `py_src/quant_figs.py`. Real: GPT-2's perplexity at each precision (cached model, cached wikitext-2), this laptop's copy bandwidth. By hand: the memory and KV numbers. Toy: the absmax and outlier examples.